Code

Brain MRI Image Segmentation Using K-Means Clustering

01 — Data Exploration and Image Representation

Keman Xiang | PhD Application Portfolio

- 1 Project Overview
  - 1.1 Background and motivation
  - 1.2 Research questions and analytical sequence
  - 1.3 Source files and provenance
- 2 Data Import
  - 2.1 Working directory and file paths
  - 2.2 Import the raw pixel matrices
  - 2.3 Validate data integrity and grayscale encoding
- 3 Image Representation and Reconstruction
  - 3.1 From a two-dimensional matrix to a pixel vector
  - 3.2 Construct the images
- 4 Exploratory Data Analysis
  - 4.1 Descriptive statistics and image dimensions
  - 4.2 Compare grayscale intensity distributions
  - 4.3 Empirical cumulative distributions
  - 4.4 Quantiles and sensitivity to zero-intensity pixels
  - 4.5 Intensity bands as a descriptive visualization
- 5 Key Findings and Implications for Subsequent Modules
- 6 Limitations and Reproducibility Notes
- 7 References and Attribution

1 Project Overview

Project: Brain MRI Image Segmentation Using K-Means Clustering
Module: 01 — Data Exploration
Language and environment: R / RStudio / R Markdown
Source: MIT OpenCourseWare, *15.071 The Analytics Edge*, Unit 6 Recitation, *Seeing the Big Picture: Segmenting Images to Create Data* (Spring 2017).

1.1 Background and motivation

Image segmentation partitions an image into regions that are comparatively homogeneous in a characteristic such as intensity, color, or texture. In medical imaging, segmentation can support analysis of anatomical structures and abnormalities. The MIT recitation illustrates an introductory approach: use pixel intensity alone as the input to unsupervised clustering of grayscale brain MRI images. This is a **teaching example of image representation and segmentation**, not a clinically validated tumor-detection system.

The recitation begins with a flower image and hierarchical clustering, then turns to brain MRI. A full pairwise-distance matrix becomes impractical for the much larger MRI image, motivating the use of K-Means. The subsequent project pratices fit clusters on a healthy MRI image and apply the learned intensity-based assignments to a second MRI image supplied as `tumor.csv`. This first module establishes the data representation and descriptive evidence needed before fitting any clustering model.

1.2 Research questions and analytical sequence

1. How are the supplied MRI images encoded as numerical data, and can they be reconstructed faithfully from the original CSV matrices?
2. What are the intensity distributions, data-quality characteristics, and image dimensions of the two images?
3. How do their overall intensity distributions differ, and what can—and cannot—be inferred from these differences?

Project sequence: 01 Data Exploration (this document) → 02 K-Means Segmentation → 03 Model Selection → 04 Cross-Image Analysis. **No clustering is fitted in Module 01.**

1.3 Source files and provenance

Supplied file	Role in this project
`Unit6_Recitation.pdf`	MIT's conceptual discussion of grayscale images, image segmentation, computational limits, and K-Means.
`Unit6_Recitation.R`	Original project workflow: read the MRI matrices, visualize, vectorize, cluster, and apply the model to a new image.
`week6_3.2.R`	Supplementary R implementation including intensity visualization, K-Means, WCSS/elbow analysis, and application to the second image.
`healthy.csv`	Original grayscale image matrix used as the model-development image in later modules.
`tumor.csv`	Original grayscale image matrix used as the second image in later modules.

Data provenance statement. All numerical analyses in this report are computed directly from the two supplied CSV files. The source materials do not provide patient demographics, MRI acquisition protocols, anatomical alignment, pixel spacing, clinical diagnoses, or pixel-level ground-truth masks. File names describe the instructional examples; they are not independently verified clinical labels. The data should not be used for diagnosis.

2 Data Import

2.1 Working directory and file paths

Read the data.

Hide

```
        find_data_dir <- function() {
          candidates <- c("data", "../data", ".", "..")
          valid <- vapply(candidates, function(p) {
            file.exists(file.path(p, "healthy.csv")) &&
              file.exists(file.path(p, "tumor.csv"))
          }, logical(1))
          if (!any(valid)) {
            stop("Original healthy.csv and tumor.csv not found. Put them in the project's data/ folde
r.")
          }
          normalizePath(candidates[which(valid)[1]], mustWork = TRUE)
        }
        data_dir <- find_data_dir()
        cat("Input directory:", data_dir, "\n")
```

```
## Input directory: /Users/mac/Downloads/MIT_Analytics_Edge/Week6/3/Tumor
```

Code explanation. `file.exists()` verifies that **both** inputs are present before proceeding. `normalizePath()` resolves the selected location. The document deliberately stops if the files are missing, rather than silently switching to example or synthetic images.

2.2 Import the raw pixel matrices

MIT's scripts use `read.csv(..., header = FALSE)` followed by `as.matrix()`. We retain this workflow because each CSV row represents a row of image pixels, not a row of patient observations; there are no variable-name headers.

Hide

```
healthy_df <- read.csv(file.path(data_dir, "healthy.csv"),
                       header = FALSE, check.names = FALSE)
tumor_df <- read.csv(file.path(data_dir, "tumor.csv"),
                     header = FALSE, check.names = FALSE)
healthy_matrix <- as.matrix(healthy_df)
tumor_matrix <- as.matrix(tumor_df)

cat("Healthy matrix:", paste(dim(healthy_matrix), collapse = " x "), "\n")
```

```
## Healthy matrix: 566 x 646
```

Hide

```
cat("Tumor matrix:  ", paste(dim(tumor_matrix), collapse = " x "), "\n")
```

```
## Tumor matrix:   571 x 512
```

Hide

```
cat("Storage modes:", storage.mode(healthy_matrix), "/",
    storage.mode(tumor_matrix), "\n")
```

```
## Storage modes: double / double
```

Output interpretation. The dimensions printed above are the actual pixel grid sizes, not counts of clinical observations. A different number of rows or columns across images is permissible for intensity-based clustering, but prevents direct pixel-to-pixel spatial comparison without image registration and resampling.

2.3 Validate data integrity and grayscale encoding

We represent grayscale intensities on a scale from **0 (black) to 1 (white)**. Before plotting or clustering, check that the supplied data are numeric, finite, nonmissing, and inside this expected range. These are data-format checks—not tests of MRI clinical validity.

Hide

```
validate_image <- function(x, label) {
  if (!is.matrix(x) || !is.numeric(x))
    stop(label, " must be a numeric matrix.")
  if (anyNA(x) || any(!is.finite(x)))
    stop(label, " contains missing or non-finite pixel values.")
  if (any(x < 0 | x > 1))
    stop(label, " has pixel intensities outside [0, 1].")
  if (nrow(x) < 2 || ncol(x) < 2)
    stop(label, " does not have a valid two-dimensional image shape.")
  TRUE
}
stopifnot(validate_image(healthy_matrix, "healthy.csv"),
          validate_image(tumor_matrix, "tumor.csv"))
cat("Both images passed numeric, finite, missingness, and [0, 1] range checks.\n")
```

```
## Both images passed numeric, finite, missingness, and [0, 1] range checks.
```

Output interpretation. A successful message means the CSV values meet the expected numerical encoding. It does **not** establish that the images share an MRI sequence, acquisition scale, registration, or clinical ground truth.

3 Image Representation and Reconstruction

3.1 From a two-dimensional matrix to a pixel vector

A grayscale image is a matrix I with r rows and c columns. Its rc entries can be rearranged into a one-dimensional vector of intensities. We use `as.vector(as.matrix(...))` ; in R, this operation traverses the matrix **column by column**. It preserves every intensity value but removes spatial coordinates from the clustering features.

Hide

```
# Convert MRI matrices into pixel intensity vectors
healthy_vector <- as.vector(healthy_matrix)
tumor_vector <- as.vector(tumor_matrix)

# Verify that vector lengths match the number of pixels
stopifnot(
  length(healthy_vector) == prod(dim(healthy_matrix)),
  length(tumor_vector) == prod(dim(tumor_matrix))
)

# Reconstruct matrices from vectors
healthy_reconstructed <- matrix(
  healthy_vector,
  nrow = nrow(healthy_matrix),
  ncol = ncol(healthy_matrix)
)

tumor_reconstructed <- matrix(
  tumor_vector,
  nrow = nrow(tumor_matrix),
  ncol = ncol(tumor_matrix)
)

# Verify that all pixel intensity values are preserved
stopifnot(
  isTRUE(all.equal(
    unname(healthy_reconstructed),
    unname(healthy_matrix)
  )),
  isTRUE(all.equal(
    unname(tumor_reconstructed),
    unname(tumor_matrix)
  ))
)

cat("Healthy pixel observations:", length(healthy_vector), "\n")
```

```
## Healthy pixel observations: 365636
```

Hide

```
cat("Tumor pixel observations:  ", length(tumor_vector), "\n")
```

```
## Tumor pixel observations:   292352
```

Data Integrity Verification. Both MRI images were converted from two-dimensional intensity matrices into one-dimensional vectors using R's column-major ordering. The vector lengths were verified against the original pixel counts, and the matrices were reconstructed to confirm that no pixel intensity values were altered during conversion. Matrix dimension names were excluded from the equality check because they are metadata rather than image intensity values. This transformation preserves the original pixel data and prepares the observations for subsequent K-Means clustering.

3.2 Construct the images

Base R's `image()` interprets matrix axes differently from a typical image file (where the first matrix row appears at the **top**). To display the CSV's first row at the top and its first column at the left, transpose the matrix and reverse its row order before calling `image()`. This is a **display transformation only**; it does not modify the source matrix or the pixel vectors used for analysis.

Hide

```
plot_grayscale <- function(m, main = "") {
  display_m <- t(m[nrow(m):1, , drop = FALSE])
  image(x = seq_len(ncol(m)), y = seq_len(nrow(m)), z = display_m,
        col = gray(seq(0, 1, length.out = 256)),
        zlim = c(0, 1), axes = FALSE, xlab = "", ylab = "",
        main = main, useRaster = TRUE, asp = 1)
  box()
}
op <- par(mfrow = c(1, 2), mar = c(3, 2, 3, 2))
plot_grayscale(healthy_matrix, "Healthy-image CSV")
plot_grayscale(tumor_matrix, "Tumor-image CSV")
```

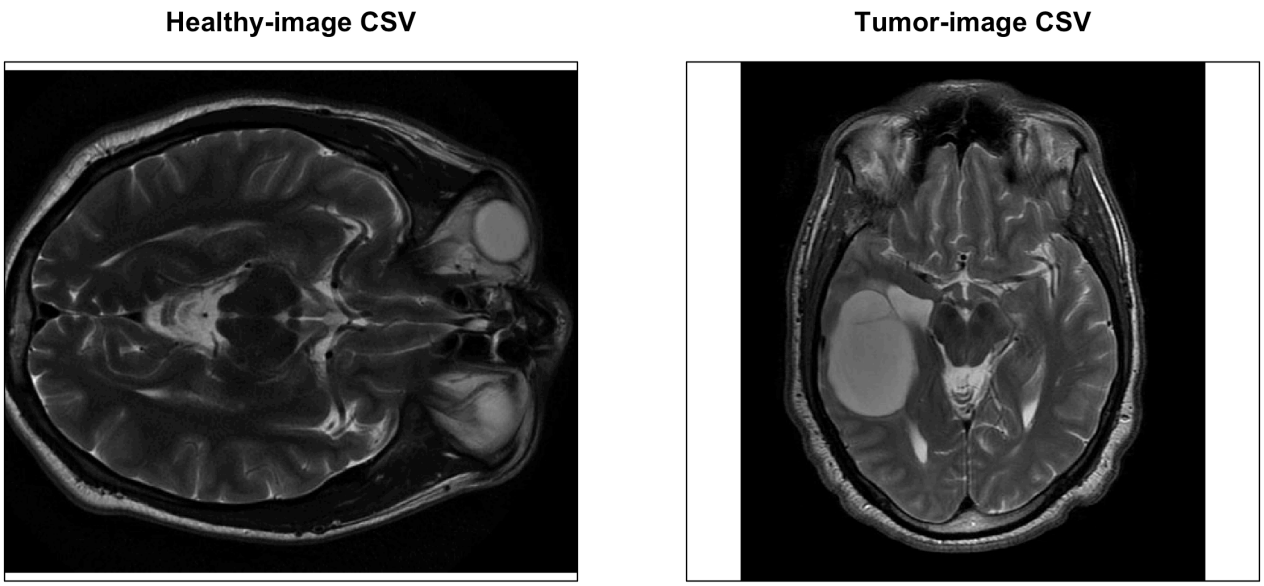


Figure 1. Grayscale reconstructions from the two original CSV pixel matrices; displayed in top-row-first orientation.

Hide

```
par(op)
```

Figure interpretation. Both panels show actual pixel intensities from the provided CSVs, with the same fixed 0–1 grayscale palette so that equal intensities receive equal shades. These are not generated MRI scans. The two images differ in pixel dimensions and appearance; visual differences alone do not establish tumor location or prove that the images are anatomically registered. Dark regions can include image background and should not automatically be interpreted as specific tissue classes.

4 Exploratory Data Analysis

4.1 Descriptive statistics and image dimensions

Summaries below are computed dynamically on the original intensity vectors. We report zero-value proportions because background pixels can strongly affect image-wide averages and clustering. The `sd()` function uses the sample standard deviation; this is a descriptive convention rather than an estimate of population-level patient variability.

Hide

```
summarize_image <- function(v, m, label) {
  data.frame(
    Image = label,
    Rows = nrow(m), Columns = ncol(m), Pixels = length(v),
    Minimum = min(v), Q1 = unname(quantile(v, .25)),
    Median = median(v), Mean = mean(v),
    Q3 = unname(quantile(v, .75)), Maximum = max(v),
    SD = sd(v), Zero_pct = 100 * mean(v == 0),
    Unique_values = length(unique(v)), Missing = sum(is.na(v)),
    check.names = FALSE
  )
}
image_summary <- rbind(
  summarize_image(healthy_vector, healthy_matrix, "Healthy"),
  summarize_image(tumor_vector, tumor_matrix, "Tumor"))
knitr::kable(image_summary, digits = 4,
             caption = "Table 1. Descriptive statistics calculated directly from the source CSV
s.")
```

Table 1. Descriptive statistics calculated directly from the source CSVs.

Image	Rows	Columns	Pixels	Minimum	Q1	Median	Mean	Q3	Maximum	SD	Zero_pct	Unique_values	Missing
Healthy	566	646	365636	0	0.0256	0.1068	0.1302	0.1795	1	0.1257	9.2950	206	0
Tumor	571	512	292352	0	0.0000	0.1333	0.1595	0.2784	1	0.1715	34.6692	254	0

Output interpretation. The healthy image contains 365,636 pixels and the tumor image contains 292,352 pixels. Their observed mean intensities are 0.1302 and 0.1595, respectively. Zero-intensity pixels account for 9.30% and 34.67%. These values describe **the images as stored**, including background; they must not be generalized to healthy and tumor-bearing brains as patient populations.

4.2 Compare grayscale intensity distributions

Histograms summarize how many pixels fall into each intensity interval. Using common breaks and a shared horizontal scale makes the two image distributions visually comparable. Because the images contain different numbers of pixels, **density** rather than raw count is plotted.

Hide

```
common_breaks <- seq(0, 1, by = 0.02)
op <- par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
hist(healthy_vector, breaks = common_breaks, freq = FALSE,
     xlim = c(0, 1), col = "gray65", border = "white",
     main = "Healthy image", xlab = "Grayscale intensity", ylab = "Density")
abline(v = mean(healthy_vector), col = "firebrick", lwd = 2, lty = 2)
hist(tumor_vector, breaks = common_breaks, freq = FALSE,
     xlim = c(0, 1), col = "gray65", border = "white",
     main = "Tumor image", xlab = "Grayscale intensity", ylab = "Density")
abline(v = mean(tumor_vector), col = "firebrick", lwd = 2, lty = 2)
```

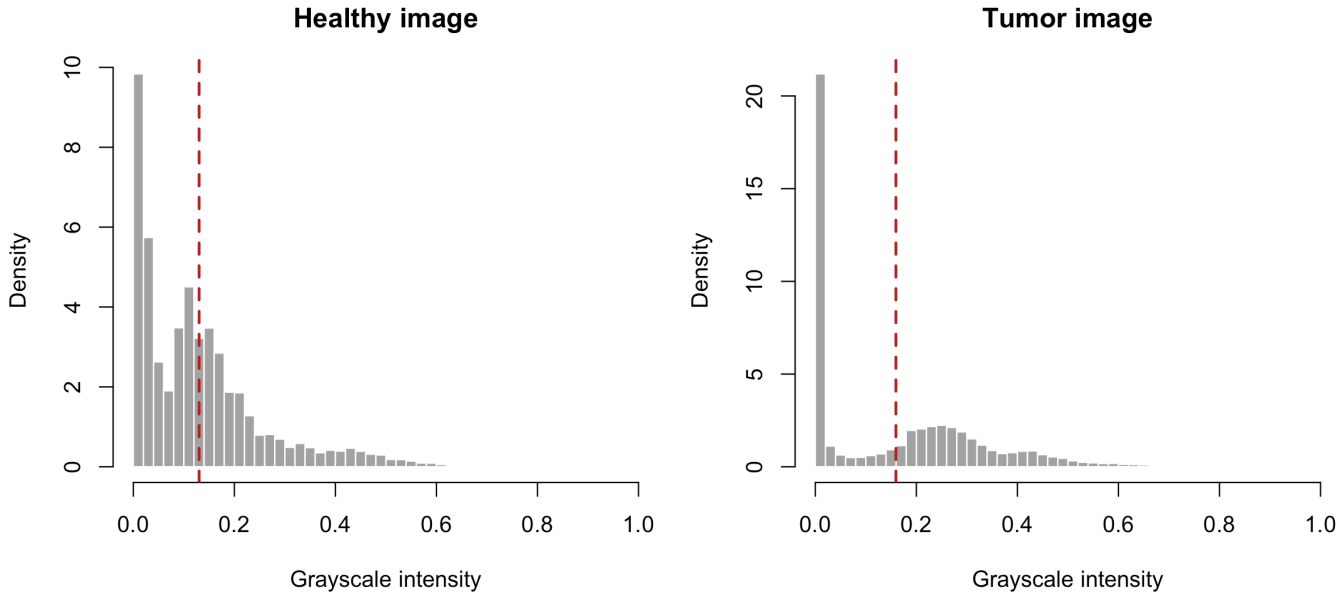


Figure 2. Pixel-intensity density histograms, using identical 0–1 bins.

Hide

```
par(op)
```

Figure interpretation. The dashed lines mark image-level mean intensities. Compare the relative concentration of near-black pixels, midrange intensities, and brighter values. A high spike at zero can reflect background rather than tissue. The observed differences are descriptive and could arise from field of view, image background, preprocessing, or acquisition settings, not necessarily disease.

4.3 Empirical cumulative distributions

An empirical cumulative distribution function (ECDF) shows the proportion of pixels with intensity less than or equal to a threshold. This makes distribution comparisons possible without selecting histogram bins.

Hide

```
plot(ecdf(healthy_vector), verticals = TRUE, do.points = FALSE,
     col = "steelblue4", lwd = 2, xlim = c(0, 1),
     main = "Empirical cumulative pixel-intensity distributions",
     xlab = "Intensity threshold", ylab = "Fraction of pixels at or below threshold")
lines(ecdf(tumor_vector), verticals = TRUE, do.points = FALSE,
      col = "darkorange3", lwd = 2)
legend("bottomright", legend = c("Healthy image", "Tumor image"),
       col = c("steelblue4", "darkorange3"), lwd = 2, bty = "n")
```

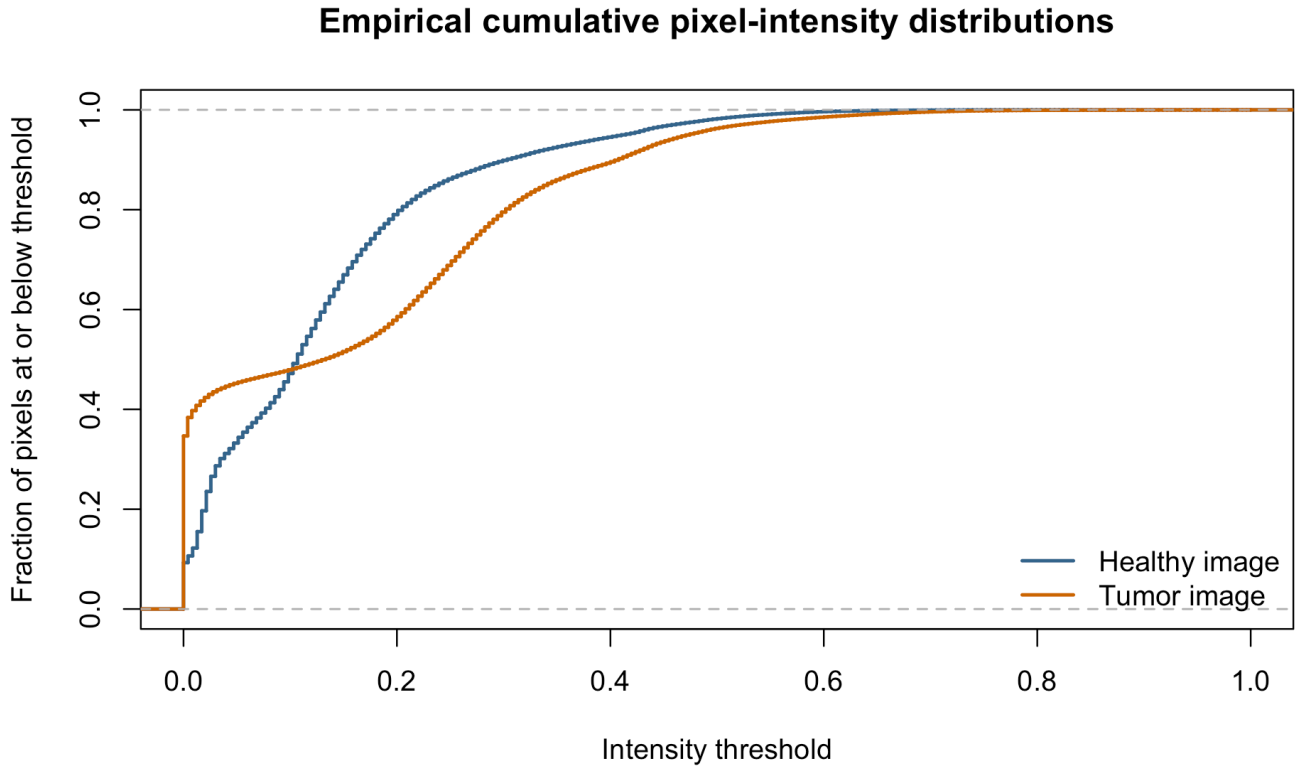


Figure 3. Empirical cumulative intensity distributions for both images.

Figure interpretation. At any intensity threshold, the vertical position of each curve gives the share of pixels no brighter than that threshold. The jump near zero quantifies the black-pixel fraction. ECDF separation indicates different *marginal* intensity distributions, not spatial localization of a tumor or a measure of diagnostic performance.

4.4 Quantiles and sensitivity to zero-intensity pixels

Zero intensity is a valid grayscale value, not a missing value. Nevertheless, an optional **descriptive sensitivity analysis** excluding exactly-zero pixels helps assess how much background influences whole-image means. This is **not** a validated brain mask: nonzero pixels can still represent background, and legitimate tissue may be dark.

Hide

```
        nonzero_summary <- data.frame(
          Image = c("Healthy", "Tumor"),
          All_pixel_mean = c(mean(healthy_vector), mean(tumor_vector)),
          Nonzero_pixel_mean = c(mean(healthy_vector[healthy_vector > 0]),
                                 mean(tumor_vector[tumor_vector > 0])),
          Nonzero_pixels = c(sum(healthy_vector > 0), sum(tumor_vector > 0)),
          Zero_pct = 100 * c(mean(healthy_vector == 0), mean(tumor_vector == 0))
        )
        knitr::kable(nonzero_summary, digits = 4,
                     caption = "Table 2. Descriptive sensitivity of mean intensity to exactly-zero pixel
s.")
```

Table 2. Descriptive sensitivity of mean intensity to exactly-zero pixels.

Image	All_pixel_mean	Nonzero_pixel_mean	Nonzero_pixels	Zero_pct
Healthy	0.1302	0.1436	331650	9.2950
Tumor	0.1595	0.2442	190996	34.6692

Output interpretation. Compare `All_pixel_mean` and `Nonzero_pixel_mean` within each image. Their difference quantifies sensitivity to the exact-zero background convention, **not** an effect of removing non-brain tissue reliably. The nonzero subsets are not guaranteed to be comparable anatomical regions across images.

4.5 Intensity bands as a descriptive visualization

For a simple, model-free visualization, divide the common intensity scale into four **fixed** bands. These thresholds are arbitrary visualization aids, not fitted clusters or anatomical tissue labels. Unlike K-Means, the cut points are not learned from the data.

Hide

```
fixed_band <- function(m) {
  matrix(as.integer(cut(as.vector(m), breaks = c(-Inf, .25, .5, .75, Inf),
                        labels = FALSE)), nrow = nrow(m), ncol = ncol(m))
}
plot_bands <- function(m, main = "") {
  z <- t(m[nrow(m):1, , drop = FALSE])
  image(seq_len(ncol(m)), seq_len(nrow(m)), z,
        col = c("#253494", "#2c7fb8", "#7fcdbb", "#ffffcc"),
        breaks = seq(.5, 4.5, by = 1),
        axes = FALSE, xlab = "", ylab = "", main = main,
        useRaster = TRUE, asp = 1)
  box()
}
op <- par(mfrow = c(1, 2), mar = c(3, 2, 3, 2))
plot_bands(fixed_band(healthy_matrix), "Healthy: fixed intensity bands")
plot_bands(fixed_band(tumor_matrix), "Tumor: fixed intensity bands")
```

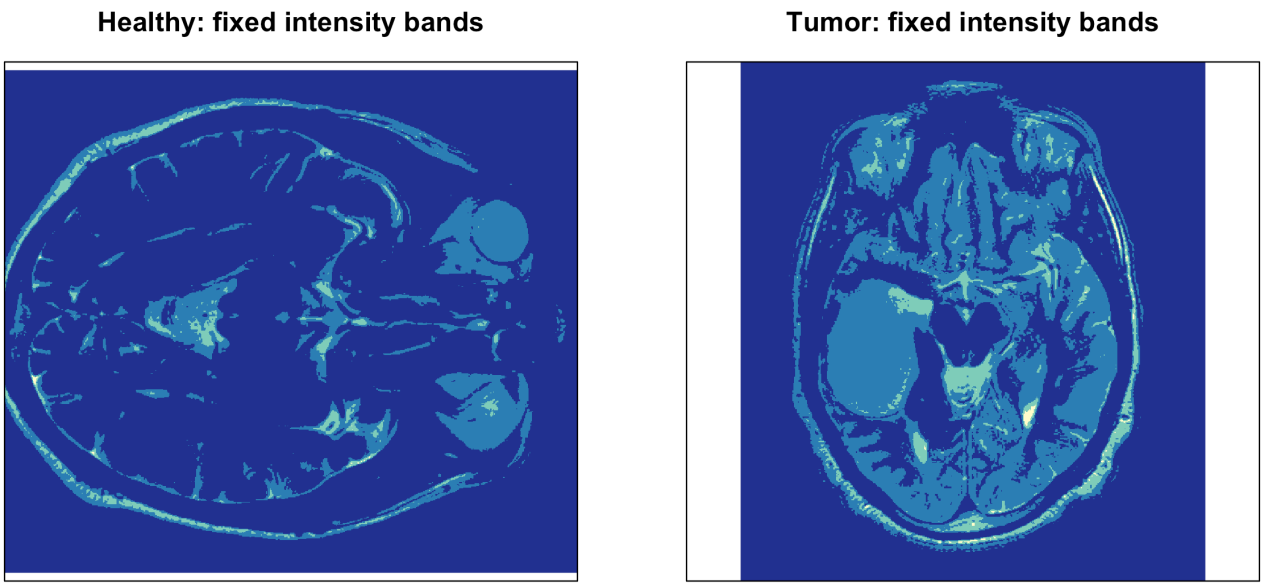


Figure 4. Four fixed grayscale-intensity bands; these are descriptive bins, not K-Means clusters or tissue labels.

Hide

```
par(op)
```

Figure interpretation. Pixels in the same fixed band share a coarse intensity interval; they need not be adjacent or represent the same tissue. The image makes clear what is lost when segmentation uses only one feature: spatial relationships, textures, and anatomical context are not explicitly modeled. Later modules will replace fixed bins with K-Means centroids learned from the healthy image.

5 Key Findings and Implications for Subsequent Modules

1. **Data representation is valid for intensity-based clustering.** Both original CSVs can be read as numeric grayscale matrices, vectorized without loss, and displayed without altering source values.
2. **The images have different shapes and distributions.** Healthy: 566 × 646; tumor: 571 × 512. Their intensity distributions should not be interpreted as aligned, paired voxel measurements.
3. **Background treatment matters.** Exactly-zero pixels represent 9.30% of the healthy image and 34.67% of the tumor image. Differences in background coverage may affect cluster proportions and model transfer.
4. **No diagnostic claims are justified.** Without clinical metadata, image registration, or expert-annotated tumor masks, this dataset cannot establish a tumor boundary or quantify detection accuracy.

Next module — `02_KMeans_Segmentation.Rmd` : fit the MIT baseline K-Means model to `healthy_vector` (original teaching setting: $k = 5$, `set.seed(1)` , `iter.max=1000`), examine cluster centroids and sizes, and reconstruct a segmented image. Formal k-selection and cross-image application are reserved for Modules 03 and 04, respectively.

6 Limitations and Reproducibility Notes

- The unit of analysis is a **pixel**, not a patient; millions of pixel-level measurements would not imply millions of independent clinical samples.
- Images are not documented as co-registered, and no scanner settings, patient attributes, or ground-truth segmentation masks were supplied.
- Grayscale intensity is a single scalar feature. Anatomically distinct areas can share intensities; identical tissues can vary with image acquisition and preprocessing.
- The fixed-band figure is exploratory and must not be interpreted as a clinical segmentation result.
- This report uses **base R** for data import and graphics and `knitr` / `rmarkdown` for reporting. It intentionally avoids external packages for segmentation at the EDA stage.

Hide

```
sessionInfo()
```

```
## R version 4.2.1 (2022-06-23)
## Platform: x86_64-apple-darwin17.0 (64-bit)
## Running under: macOS Big Sur ... 10.16
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.2/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.2/Resources/lib/libRlapack.dylib
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## loaded via a namespace (and not attached):
##  [1] digest_0.6.33     R6_2.5.1          lifecycle_1.0.4   jsonlite_1.8.8
##  [5] evaluate_0.23     highr_0.10        cachem_1.0.8      rlang_1.1.2
##  [9] cli_3.6.2         rstudioapi_0.15.0 jquerylib_0.1.4   bslib_0.6.1
## [13] rmarkdown_2.25    tools_4.2.1       xfun_0.41         yaml_2.3.8
## [17] fastmap_1.1.1     compiler_4.2.1    htmltools_0.5.7   knitr_1.45
## [21] sass_0.4.8
```

7 References and Attribution

1. MIT OpenCourseWare (2017). *15.071 The Analytics Edge*, Unit 6 Recitation: **Seeing the Big Picture — Segmenting Images to Create Data**. Course slides `Unit6_Recitation.pdf` .
2. MIT course companion script supplied as `Unit6_Recitation.R` , particularly Videos 4–6.

3. Supplementary course exercise supplied as `week6_3.2.R`.
4. Original course data supplied as `healthy.csv` and `tumor.csv`.

Project statement: This portfolio module extends the supplied original project workflow with explicit validation, reproducible descriptive statistics, orientation-aware plotting, comparative exploratory analysis, and methodological limitations. All new analyses are presented as exploratory extensions, not as results claimed by MIT.